

LLMsVerifier_User_Guide

LLMsVerifier Integration User Guide

Version: 1.0.0
Date: 2026-04-30
Authority: CONST-036 through CONST-041

Table of Contents

- 1. [Overview](#)
 - 2. [Architecture](#)
 - 3. [Configuration](#)
 - 4. [Environment Variables](#)
 - 5. [API Endpoints](#)
 - 6. [CLI Usage](#)
 - 7. [Provider Integration](#)
 - 8. [Troubleshooting](#)
 - 9. [Challenge Verification](#)
-

Overview

LLMsVerifier is the **single source of truth** for all model and provider metadata in HelixCode. It provides:

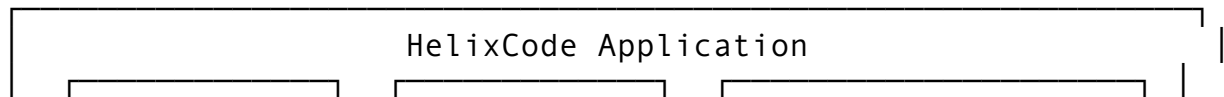
- **Model Discovery:** Live model listings from 15+ providers
- **Verification Scores:** Real-time quality scores across 5 dimensions
- **Provider Health:** Circuit-breaker protected health monitoring
- **Capability Flags:** Dynamic capability detection (vision, tools, streaming, reasoning)
- **Fallback Models:** 7 constitutional fallback models when verifier is unavailable

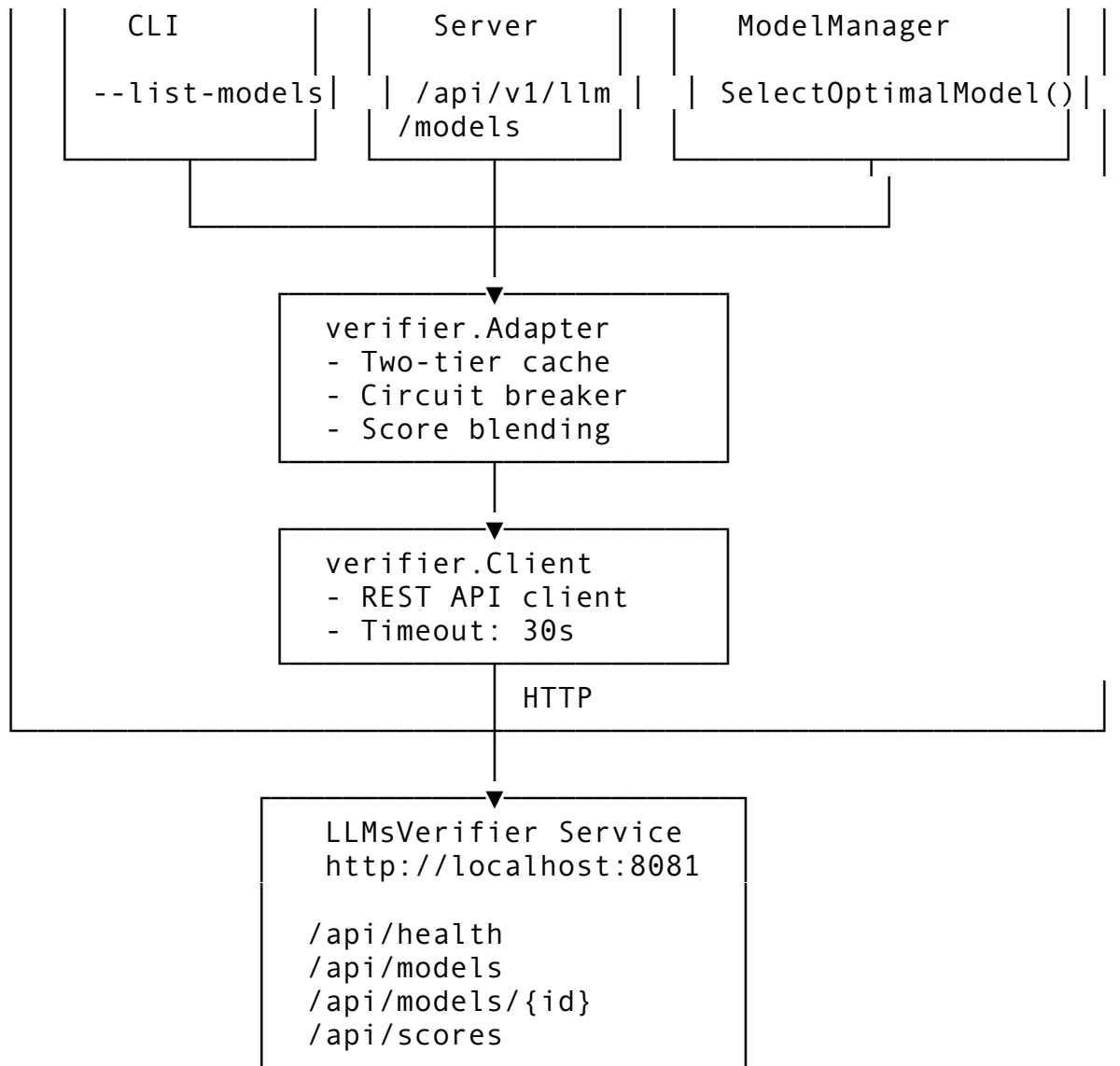
Why This Matters (Anti-Bluff)

Before this integration, model lists were **hardcoded** in multiple places: - CLI showed 3 fake models - Server API returned static JSON - Provider adapters hardcoded `SupportsVision: true/false`

Now, every model displayed to users comes from the verifier (or its cached replica). If the verifier is down, the constitutional fallback list ensures the system remains functional.

Architecture





Components

Component	File	Purpose
REST Client	<code>internal/verifier/client.go</code>	HTTP client for verifier API
Adapter	<code>internal/verifier/adapter.go</code>	Score bridge + cache interface
Cache	<code>internal/verifier/cache.go</code>	Two-tier in-memory cache
Health Monitor	<code>internal/verifier/health.go</code>	Circuit breaker
Poller	<code>internal/verifier/poller.go</code>	Background update goroutine
Bootstrap	<code>internal/verifier/bootstrap.go</code>	Application startup helper
Bridge		Provider enrichment

Component	File	Purpose
	internal/llm/ verifier_bridge.go	

Configuration

Minimal Config (configs/verifier.yaml)

```
verifier:
  enabled: true
  mode: remote
  endpoint: http://localhost:8081
  timeout: 30s
  cache_ttl: 5m
  polling_interval: 60s

  scoring:
    weights:
      code_capability: 0.40
      responsiveness: 0.20
      reliability: 0.20
      feature_richness: 0.15
      value_proposition: 0.05
      min_acceptable_score: 6.0

  health:
    failure_threshold: 5
    recovery_threshold: 3
    circuit_breaker:
      enabled: true
      half_open_timeout: 60s
```

Provider-Specific Config

```
providers:
  openai:
    enabled: true
    endpoint: https://api.openai.com/v1
    models:
      - gpt-4o
      - gpt-4o-mini
  ollama:
    enabled: true
    endpoint: http://localhost:11434
    models:
      - llama-3.2-3b
```

Environment Variables

Variable	Default	Description
HELIX_VERIFIER_ENABLED	false	Master enable/disable
HELIX_VERIFIER_ENDPOINT	http://localhost:8081	Verifier REST API URL
HELIX_VERIFIER_API_KEY	" "	Optional auth key
HELIX_VERIFIER_TIMEOUT	30s	Request timeout
HELIX_VERIFIER_CACHE_TTL	5m	Cache TTL
HELIX_VERIFIER_POLLING_INTERVAL	60s	Background poll interval
HELIX_VERIFIER_MIN_SCORE	6.0	Minimum acceptable score
OPENAI_API_KEY	" "	OpenAI provider key
ANTHROPIC_API_KEY	" "	Anthropic provider key
GEMINI_API_KEY	" "	Gemini provider key
DEEPSEEK_API_KEY	" "	DeepSeek provider key
GROQ_API_KEY	" "	Groq provider key
MISTRAL_API_KEY	" "	Mistral provider key
XAI_API_KEY	" "	xAI provider key
OPENROUTER_API_KEY	" "	OpenRouter provider key

API Endpoints

List Models (Verifier-Aware)

```
curl http://localhost:8080/api/v1/llm/models
```

Response (verifier available):

```
{
  "status": "success",
  "models": [
    {
      "id": "gpt-4o",
      "name": "GPT-4o",
      "provider": "openai",
      "context_length": 128000,

```

```

        "score": 9.1,
        "tier": 1,
        "verified": true,
        "status": "available",
        "supports_vision": true,
        "supports_tools": true,
        "source": "verifier"
    }
],
"count": 1,
"source": "verifier",
"last_updated": "2026-04-30T20:00:00Z"
}

```

Response (verifier unavailable):

```

{
  "status": "success",
  "models": [...],
  "source": "fallback",
  "last_updated": "2026-04-30T20:00:00Z"
}

```

List Providers

```
curl http://localhost:8080/api/v1/llm/providers
```

CLI Usage

List Models

```
./helixcode --list-models
```

Output uses verifier data with 3-tier fallback: 1. **Verifier** — live model list from LLMsVerifier 2. **Provider** — Ollama /api/tags or equivalent 3. **Fallback** — 7 constitutional fallback models

Select Optimal Model (Internal)

The `ModelManager.SelectOptimalModel()` blends: - **60%** verifier score (normalized 0-10) - **40%** local heuristic (capability, context, task, hardware)

Task-specific boosts: - Code generation: +15% from verifier `code_capability` score - Planning/Analysis: +10% from verifier `reliability` score

Provider Integration

Supported Providers (Minimum Set)

Provider	Type	Auth Required
OpenAI	Cloud	API Key
Anthropic	Cloud	API Key
Gemini	Cloud	API Key
DeepSeek	Cloud	API Key
Groq	Cloud	API Key
Mistral	Cloud	API Key
xAI	Cloud	API Key
OpenRouter	Cloud	API Key
Ollama	Local	None
Llama.cpp	Local	None

Adding a New Provider

1. Create `internal/llm/<name>_provider.go`
 2. Implement the Provider interface
 3. Do NOT hardcode `SupportsVision` or `Capabilities`
 4. Call `EnrichModelInfo(&model)` after building model list
 5. Add config to `configs/verifier.yaml`
 6. Add env var binding to `internal/config/config.go`
 7. Write integration test
-

Troubleshooting

“Verifier unavailable” Warning

⚠ Verifier unavailable (connection refused), using fallback...

Cause: LLMsVerifier service is not running at the configured endpoint.

Fix:

```
# Start the verifier service (see LLMsVerifier repository)
cd LLMsVerifier && go run ./cmd/server
```

```
# Or disable verifier in config
export HELIX_VERIFIER_ENABLED=false
```

Empty Model List

Cause: All providers disabled or no API keys configured.

Fix:

```
export OPENAI_API_KEY=sk-...  
export ANTHROPIC_API_KEY=sk-ant-...
```

Circuit Breaker Open

⚠ Verifier circuit breaker OPEN – too many failures

Cause: 5+ consecutive failures to the verifier.

Fix: Check verifier health. The circuit will auto-recover after `half_open_timeout`.

Challenge Verification

Run all verifier challenges:

```
# Unit tests (mocks OK)  
go test -v ./internal/verifier/...  
  
# Integration tests (real HTTP server)  
go test -tags=integration -v ./internal/verifier/...  
  
# Hardcode check (CONST-037)  
bash scripts/anti_bluff/verifier_hardcode_check.sh  
  
# Capability check (CONST-041)  
bash scripts/anti_bluff/verifier_capability_check.sh  
  
# Full E2E challenge  
bash scripts/anti_bluff/verifier_e2e_challenge.sh  
  
# Makefile targets  
make test-verifier-unit  
make test-verifier-integration  
make test-verifier-challenges
```

References

- [Constitution](#) — CONST-036 through CONST-041
- [AGENTS.md](#) — Agent guidelines for verifier work
- [CLAUDE.md](#) — Constitutional mandates
- Integration Plan: `docs/llms_verifier/LLMsVerifier_HelixCode_Integration_Plan.md`

Sources verified

Sources verified 2026-05-29: <https://github.com/ollama/ollama/blob/main/docs/api.md> (Ollama default endpoint <http://localhost:11434> confirmed — matches the guide's example) ; <https://go.dev/doc/devel/release> (Go 1.26.3 latest stable; matches inner go.mod go 1.26) — the provider roster (OpenAI, Anthropic, Gemini, DeepSeek, Groq, Mistral, xAI, OpenRouter, Ollama, Llama.cpp) and the per-provider *_API_KEY env vars are sourced at runtime from LLMsVerifier per CONST-036/039 and are NOT hardcoded version-bearing instructions, so no static endpoint pins required correction. Project version authority is go.mod + CLAUDE.md §3.1. Negative findings: OpenAI's hosted API-reference page (platform.openai.com) returned HTTP 403 to automated fetch and the openai-openapi spec file on GitHub is too large to render via fetch, so the <https://api.openai.com/v1> base URL shown in the guide could not be machine-confirmed this run (it is a long-stable, widely-used endpoint; flagged here for honesty, not corrected).